

AI Assisted Coding Lab ass-13.4

Name : G.Rakesh .Rao

H.T.No:2303A51851

Task 1: Refactoring Data Transformation Logic

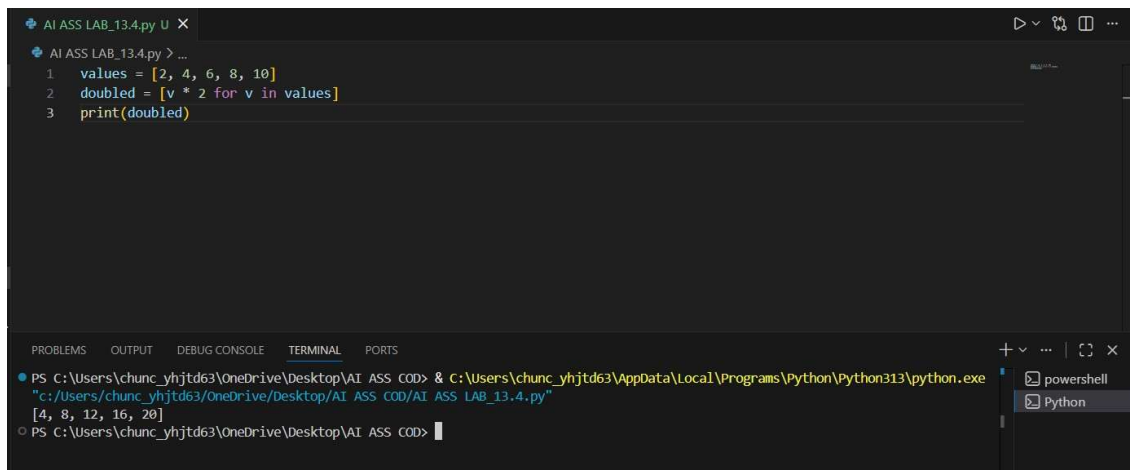
Task Description

- Review the legacy code that computes transformed values using an explicit loop.
- Use an AI tool to suggest a more Pythonic refactoring approach.
- Refactor the code using list comprehensions or helper functions while preserving the output.

PROMPT:

Refactor the following Python code to make it shorter and more Pythonic using list comprehension while keeping the same output.

```
values = [2, 4, 6,  
8, 10]  
doubled =  
[]  
for v in values:  
    doubled.append(  
        v * 2)  
print(doubled)
```



```
AI ASS LAB_13.4.py X
AI ASS LAB_13.4.py > ...
1 values = [2, 4, 6, 8, 10]
2 doubled = [v * 2 for v in values]
3 print(doubled)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\chunc_yhjt63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjt63\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/chunc_yhjt63/OneDrive/Desktop/AI ASS COD/AI ASS LAB_13.4.py"
[4, 8, 12, 16, 20]
PS C:\Users\chunc_yhjt63\OneDrive\Desktop\AI ASS COD>
```

Task 2: Improving Text Processing Code Readability

- Analyze the legacy code that constructs a sentence using repeated string concatenation.
- Ask AI to suggest a more efficient and readable approach.
- Refactor the code accordingly while keeping the final output unchanged.

PROMPT:

Refactor the following Python code to improve readability and efficiency by avoiding repeated string concatenation. Keep the same output.



```
AI ASS LAB_13.4.py X
AI ASS LAB_13.4.py > ...
1
2
3 words = ["Refactoring", "with", "AI", "improves", "quality"]
4 message = ".join(words)
5 print(message)
```

OUTPUT:

```
PS C:\Users\chunc_yhjt63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjt63\AppData\Local\Programs\Python\Python313\python.exe  
"c:/Users/chunc_yhjt63/OneDrive/Desktop/AI ASS COD/AI ASS LAB_13.4.py"  
Refactoring with AI improves quality  
PS C:\Users\chunc_yhjt63\OneDrive\Desktop\AI ASS COD> █
```

Task 3: Safer Access to Configuration Data

- Review the legacy code that manually checks for dictionary keys.
- Use AI suggestions to refactor the code using safer dictionary access methods.
- Ensure the behavior remains the same for missing keys.

PROMPT:

Refactor the following Python code to use a safer dictionary access method instead of manually checking keys. Keep the same output if the key is missing.

```
config = {"host": "localhost", "port":
```

```
8080} if "timeout" in config:
```

```
    print(config["timeout"])
```

```
else:    print("Default
```

```
timeout used")
```

```
AI ASS LAB_13.4.py X  
AI ASS LAB_13.4.py > ...  
1  config = {"host": "localhost", "port": 8080}  
2  print(config.get("timeout", "Default timeout used"))
```

OUTPUT:

```
PS C:\Users\chunc_yhjt63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjt63\AppData\Local\Programs\Python\Python313\python.exe  
"c:/Users/chunc_yhjt63/OneDrive/Desktop/AI ASS COD/AI ASS LAB_13.4.py"  
Default timeout used
```

Task 4: Refactoring Conditional Logic for

Scalability

Examine the multiple if–elif conditions used to determine operations.

Ask AI to suggest a cleaner, scalable alternative.

Refactor the logic using mapping techniques

while preserving functionality.

PROMPT:

Refactor the following Python code to replace multiple if–elif conditions with a cleaner and scalable mapping approach while keeping the same output.

```
AI ASS LAB_13.4.py U
AI ASS LAB_13.4.py > action
1  action = "divide"
2  x, y = 10, 2
3
4  operations = {
5      "add": lambda a, b: a + b,
6      "subtract": lambda a, b: a - b,
7      "multiply": lambda a, b: a * b,
8      "divide": lambda a, b: a / b,
9  }
10
11  result = operations.get(action, lambda a, b: None)(x, y)
12  print(result)
```

OUTPUT:

```
PS C:\Users\chunc_yhjt63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjt63\AppData\Local\Programs\Python\Python313\python.exe
"c:/Users/chunc_yhjt63/OneDrive/Desktop/AI ASS COD/AI ASS LAB_13.4.py"
5.0
PS C:\Users\chunc_yhjt63\OneDrive\Desktop\AI ASS COD> |
```

Task 5: Simplifying Search Logic in Collections

Identify the explicit loop used for searching an item.

Use AI assistance to refactor the logic into a

more concise and readable form.

Maintain the same output behavior.

PROMPT:

Refactor the following Python code to simplify the search logic in a collection using a more concise Python approach while keeping the same output.

```
AI ASS LAB_13.4.py U X
AI ASS LAB_13.4.py > inventory
1 inventory = ["pen", "notebook", "eraser", "marker"]
2 found = "eraser" in inventory
3 print("Item Available" if found else "Item Not Available")
```

OUTPUT:

```
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> & C:\Users\chunc_yhjtd63\AppData\Local\Programs\Python\Python313\python.exe
"c:/Users/chunc_yhjtd63/OneDrive/Desktop/AI ASS COD/AI ASS LAB_13.4.py"
Item Available
PS C:\Users\chunc_yhjtd63\OneDrive\Desktop\AI ASS COD> |
```